Digital Root of a Number

Using Recursion and a Static Variable

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a recursive function to calculate the digital root of a number. Use a static variable to count the number of recursive calls made by the function.

Example: The digital root of 9875 is 2.

```
9875  =>  9 + 8 + 7 + 5 = 29
29    =>  2 + 9 = 11
11    =>  1 + 1 = 2
```

2. Objectives

- To understand and apply recursion (a function calling itself) to solve a problem.
- To identify the base case and the recursive case of a recursive function.
- To understand the use of a static variable, which retains its value between function calls.
- To write, compile and test a complete C program.

3. Theory

3.1 Digital Root

The digital root of a non-negative number is the single-digit value obtained by repeatedly adding the digits of the number until only one digit remains. For example, the digits of 9875 add up to 29, the digits of 29 add up to 11, and the digits of 11 add up to 2. Since 2 is a single digit, it is the digital root.

3.2 Recursion

Recursion is a technique in which a function calls itself to solve a smaller version of the same problem. Every recursive function needs two parts:

- **Base case:** the condition that stops the recursion. Here, if the number is less than 10 (a single digit), that number is the answer.
- **Recursive case:** the function calls itself with a smaller problem. Here, it finds the sum of the digits and finds the digital root of that sum.

3.3 Static Variable

A variable declared with the keyword static is created only once and keeps its value for the entire run of the program. Unlike an ordinary local variable, it is not re-created each time a function is called. This is why it is suitable for counting how many times a recursive function has called itself. In this program the static variable recursiveCalls is increased by 1 just before every recursive call.

4. Algorithm

1. Start.
2. Read the number n from the user. If n is negative, make it positive.
3. Set the static counter recursiveCalls to 0.
4. Call digitalRoot(n).

5. In digitalRoot(n): if n < 10, return n (base case).
6. Otherwise, find the sum of all digits of n (using n % 10 and n / 10).
7. Increase recursiveCalls by 1.
8. Return digitalRoot(sum) (recursive call), and go to step 5.
9. Display the digital root and the value of recursiveCalls, then stop.

5. Source Code

File name: `digital_root.c`

```c
#include <stdio.h>

/* Static variable: keeps its value between function calls.
   It counts how many times digitalRoot() calls itself. */
static int recursiveCalls = 0;

/* Recursive function to find the digital root of a number */
int digitalRoot(long long n)
{
    long long sum = 0, temp = n;

    /* Base case: a single-digit number is its own digital root */
    if (n < 10)
        return (int)n;

    /* Add all the digits of n */
    while (temp > 0)
    {
        sum += temp % 10;
        temp /= 10;
    }

    printf("   %lld => %lld\n", n, sum);

    /* Count this recursive call, then recurse on the digit sum */
    recursiveCalls++;
    return digitalRoot(sum);
}

int main(void)
{
    long long num;
    int root;

    printf("Enter a number: ");
    scanf("%lld", &num);

    if (num < 0)          /* use the positive value */
        num = -num;

    recursiveCalls = 0;   /* reset the counter before every run */

    printf("\nSteps:\n");
    root = digitalRoot(num);

    printf("\nDigital root of %lld is %d\n", num, root);
    printf("Number of recursive calls = %d\n", recursiveCalls);

    return 0;
}
```

6. Working of the Program (Dry Run)

Dry run of the program for the input 9875:

Call	Value of n	Is n < 10?	Digit sum	recursiveCalls
1	9875	No	9+8+7+5 = 29	1
2	29	No	2+9 = 11	2
3	11	No	1+1 = 2	3
4	2	Yes: returns 2	-	3

The function is called 4 times in total (the first call from main() and 3 recursive calls). The static variable stores 3, the number of times the function called itself. The value 2 is returned back through all the calls to main().

7. Output

Sample Output (Input: 9875)

```
Enter a number: 9875

Steps:
   9875 => 29
   29 => 11
   11 => 2

Digital root of 9875 is 2
Number of recursive calls = 3
```

Test Cases

Input	Digital Root	Recursive Calls	Result
9875	2	3	Correct
12345	6	2	Correct
999999999	9	2	Correct
9	9	0	Correct
0	0	0	Correct

8. Conclusion

The digital root of a number was successfully calculated using a recursive function in C. The base case (n < 10) stops the recursion, and every other call reduces the problem by replacing the number with the sum of its digits. A static variable was used to count the recursive calls, since it keeps its value across all the calls. For the example 9875, the program correctly gives the digital root 2 with 3 recursive calls. The program was compiled and tested with several inputs and works correctly.